

Lecture 7 & 8: Understanding Requirements

1. Introduction to Requirements

Definition (IEEE Standard 729):

- A condition or capability needed by a user to solve a problem or achieve an objective.
- A condition or capability that must be met by a system to satisfy a contract, standard, or specification.

2. Requirements Engineering (RE)

- The broad spectrum of tasks & techniques that lead to an understanding of requirements.
- Begins with **communication** and continues into the **modeling** activity.
- Bridges the gap between understanding needs and design/construction.
- RE involves: finding out, analysing, documenting, and checking services & constraints.

Why RE Matters:

- Jumping straight to coding without clear requirements is a common but flawed approach.
- Lack of RE leads to failed software projects.

3. Levels of Abstraction

User Requirements:

- Written in natural language + diagrams.
- States what the system should provide and the constraints it must operate under.
- May range from broad statements to detailed descriptions.

Example: "The library system should allow a student to borrow a book."

System Requirements:

- Precise specification of functions, services, and operational constraints.
- Also called a **functional specification**.
- May form part of the contract between buyer and developer.

Example: "The system provides a withdraw interaction requiring ISBN & copy number; fails if book is already withdrawn, 5-book limit exceeded, debt >\$5, etc."

4. Stakeholders

- Anyone affected by or with a legitimate interest in the system.
- Range from end-users and managers to external regulators.

Readers of User Requirements:

- Client managers, System end-users, Client engineers, Contractor managers, System architects.

Readers of System Requirements:

- System end-users, Client engineers, System architects, Software developers.

Mentcare System Stakeholders (Example):

- Patients & their relatives
- Doctors – assess and treat patients
- Nurses – coordinate consultations & administer treatments
- Medical receptionists – manage appointments
- IT staff – install & maintain the system
- Medical ethics manager – ensures ethical guidelines are met
- Health care managers – obtain management information
- Medical records staff – maintain & preserve records

5. Types of Requirements

A. Functional Requirements (FR)

- Statements of services the system should provide.
- Define how the system reacts to inputs and behaves in situations.
- Can also state what the system should *not* do.

Mentcare Examples:

- Users can search appointment lists for all clinics.
- System generates daily patient attendance lists per clinic.
- Staff are identified by a unique 8-digit employee number.

Requirements Completeness & Consistency:

- **Completeness** – all required services and information are defined.
- **Consistency** – requirements must not contradict each other.
- Difficult to achieve fully in large, complex systems with many stakeholders.
- Inconsistencies may only surface during deeper analysis or development.

B. Non-Functional Requirements (NFR)

- Constraints on services or functions – timing, standards, development process.
- Apply to the system *as a whole*, not individual features.

NFR Categories:

- **Product Requirements:** Specify how the delivered product must behave (speed, reliability, usability).
- **Organizational Requirements:** Consequences of organizational policies and procedures.
- **External Requirements:** Arise from factors external to the system, e.g. legislation.

Specific NFR Types:

- **Reliability:** Concerns failure frequency. E.g., heart-monitor must fail less than once in 20 years.
- **Efficiency:** Hardware resources needed – processing (MIPS/MHz), storage (MB/GB), communication (KBPS).
- **Portability:** Adaptation of software to different hardware/OS environments.

- **Interoperability:** Extent to which systems can exchange and interpret shared data.
- **Integrity:** Prevents unauthorized access. E.g., GIS read-only public access with restricted documents.
- **Usability:** Staff training time and ease of operation. E.g., staff handles 60 calls/day; training $\leq$ 2 days.

6. Specifying Requirements

- Use **natural language, tables, and diagrams** for broad understandability.
- Both functional and non-functional requirements must be accessible to non-technical users.

Guidelines for Writing Requirements:

- Invent a standard format and use it consistently.
- Use language consistently throughout.
- Avoid computer jargon.
- Separate functional and non-functional requirements.
- Distinguish requirement priorities using **MoSCoW** (Must / Shall / Could / Want – no TBDs).
- Make requirements testable – write test cases.

7. Word Processor Spell Checker – Full Example

User Requirement:

- Finding spelling errors and replacing misspelled words chosen from a suggested list.

Functional Requirements:

- Spell checker finds and highlights misspelled words.
- Displays a dialog box with suggested replacements.
- User selects a replacement from the list.
- System replaces the misspelled word with the selected word.

Non-Functional Requirement:

- Must integrate into the existing word processor running on the Windows platform.

8. Common Problems in Requirements Writing

- **Difficult to read** – overly complex sentences.
- **Ambiguity** – unclear terms, e.g. "5% of what?"
- **Incompleteness** – missing cases, e.g. what if sales are above target?

Reading Assignment

- Chapter 5: Understanding Requirements – *Software Engineering: A Practitioner's Approach*
- Resource link: <https://sites.google.com/cuilahore.edu.pk/sec>